

Python Fundamentals — Assignment

Based on: Variables, Objects, Loops, Conditions, Functions & Scope

Topic 1: Variables & Dynamic Typing

Reference syntax / example:

```
x = 3          # int
x = 4.5        # same variable, now a float — no error!

# Python has no type declarations (unlike C/Java):
# int x = 3;    <-- this is C, NOT Python
```

Practice Questions:

Q1.1. Write a variable `age` and assign it the integer 25. Print its value and its type using `type()`.

Q1.2. Assign the string "10" to a variable called `num`, then reassign `num` to the integer 10. Print `num` and `type(num)` after each assignment to show dynamic typing in action.

Q1.3. In C or Java, why would `int x = 3; x = 4.5;` fail to compile, but the equivalent code runs fine in Python? Answer in 2-3 sentences.

Q1.4. Predict the output (write your prediction, then run it):

```
x = 5
```

```
x = "five"
```

```
print(x)
```

Topic 2: Common Data Types

Reference syntax / example:

```
a = 10          # int
b = 3.14        # float
c = 2 + 3j      # complex
d = [1, 2, 3]   # list
e = (1, 2, 3)   # tuple
f = range(5)    # range
g = True        # bool
h = "hello"     # string
```

Practice Questions:

Q2.1. Create one variable of each type: int, float, string, bool, and list. Print each variable along with its type().

Q2.2. What is the difference between a list and a tuple? Write one line of code that creates each, and one line that shows a list can be changed after creation while a tuple cannot.

Q2.3. Given price = 499.99 and in_stock = True, write an if statement that prints "Available" only when in_stock is True.

Topic 3: Objects, Identity (is) vs Equality (==)

Reference syntax / example:

```
>>> x = 3
>>> y = 3.0
>>> x is y
False          # different objects in memory

>>> x == y
True           # same value
```

Practice Questions:

Q3.1. Explain in your own words the difference between `is` and `==` in Python.

Q3.2. Predict the output, then verify by running it:

```
a = [1, 2, 3]
```

```
b = [1, 2, 3]
```

```
print(a == b)
```

```
print(a is b)
```

Q3.3. Create two variables `x = 100` and `y = 100`. Check `x is y` and `x == y`. Now create `p = 1000` and `q = 1000` and check the same. Look up (or discuss) why small integers may behave differently from large ones.

Q3.4. Everything in Python is an object. Using `type(5)`, `type("hi")`, and `type([1,2])`, show that even numbers and strings are objects with a type.

Topic 4: Strings & Methods

Reference syntax / example:

```
>>> x = "Hello"
>>> x.lower()
'hello'

# general syntax:
# <var_name>.<method_name>(params)

>>> x                # calling a method does NOT change x itself
'Hello'              # unless you reassign: x = x.lower()
```

Practice Questions:

Q4.1. Create a string `name = "pYTHOn"`. Print `name.upper()`, `name.lower()`, and `name.title()`.

Q4.2. Predict the output, then run it:

```
s = "Hello"
```

```
s.lower()
```

```
print(s)
```

Q4.3. Now try:

```
s = "Hello"
```

```
s = s.lower()
```

```
print(s)
```

Explain why the output differs from the previous question.

Q4.4. Given `sentence = " Learning Python is fun "`, use string methods to: (a) remove the leading/trailing spaces, (b) count how many times the letter 'n' appears, (c) split the sentence into a list of words.

Topic 5: Loops — `for`, `range()`, `while`

```
for i in range(0, 10):    # Python uses indentation, not { }
    print(i)

# range(start, stop[, step])
for i in range(2, 12, 3):
    print(i)              # 2, 5, 8, 11

i = 2
while i < 12:
    print(i)
    i += 3
```

Reference syntax / example:

Practice Questions:

Q5.1. Write a for loop using range() that prints the numbers 1 through 10 (inclusive).

Q5.2. Write a for loop that prints every 3rd number from 0 to 30 using the step argument of range().

Q5.3. Predict the output, then run it:

```
for i in range(10, 0, -2):
```

```
    print(i)
```

Q5.4. Rewrite this for loop as a while loop that produces the same output:

```
for i in range(0, 20, 4):
```

```
    print(i)
```

Q5.5. How many times does the condition $i < 10$ get evaluated in the loop below? Explain your reasoning before running it.

```
i = 0
```

```
while i < 10:
```

```
    print(i)
```

```
    i += 1
```

Topic 6: Conditions — if / elif / else, modulo (%)

```
# % (modulo) returns the remainder
# 22 % 3 is 1 because 22 / 3 = 21 remainder 1

for i in range(0, 5):
    if i % 2 == 0:
        print(i)
    else:
        print(i + 10)
```

Practice Questions:

Q6.1. Write a program that loops through numbers 1 to 20 and, using `% 2 == 0`, prints only the even numbers.

Q6.2. Write a program that loops through numbers 1 to 15 and prints "Fizz" if the number is divisible by 3, "Buzz" if divisible by 5, "FizzBuzz" if divisible by both, and the number otherwise. (Hint: check `% 15` first, or check both conditions with `elif`.)

Q6.3. Predict the output, then run it:

```
for i in range(0, 5):

    if i % 3 == 0:

        print(i)

    elif i % 3 == 1:

        print(i + 10)

    else:

        print(i - 10)
```

Q6.4. Write a program that asks whether a number entered by the user is positive, negative, or zero using `if / elif / else`.

Topic 7: Functions & Passing Arguments

```
def my_abs(val):  
    if val < 0:  
        return 0 - val  
    return val  
  
print(my_abs(-7))    # 7  
  
def inc_val(val):  
    val = val + 1     # only changes the LOCAL copy  
  
x = 7  
inc_val(x)  
print(x)             # still 7 - x was passed by value/reference-to-object
```

Practice Questions:

Q7.1. Write a function `my_abs(val)` that returns the absolute value of `val` without using Python's built-in `abs()`. Test it with a positive number, a negative number, and 0.

Q7.2. What will `my_abs("Hi")` do when run, given the function above? Predict the error, then run it to confirm.

Q7.3. Write a function `print_abs(val)` that prints (not returns) the absolute value. Then run `x = print_abs(-2.7)`; `print(x)`. What gets printed for `x`, and why?

Q7.4. Predict the output, then run it:

```
def inc_val(val):  
    val = val + 1  
  
x = 7  
inc_val(x)  
print(x)
```

Explain in your own words why reassigning `val` inside the function does not change `x`.

Q7.5. Write a function `swap_message(val1, val2)` that swaps the two LOCAL variables inside the function and prints them. Then call `swap_message(6, 3)` and separately print the original variables `x = 6, y = 3` after the call to show that the originals are unaffected.

Topic 8: Scope — Local vs Global Variables

```
def my_abs(val):  
    if val < 0:  
        return 0 - val  
    return val  
  
print(val)      # NameError: name 'val' is not defined  
                # val only exists inside my_abs()  
  
my_val = 0      # global variable  
def show():  
    print(my_val)  # can READ a global from inside a function
```

Practice Questions:

Q8.1. Predict the output, then run it:

```
def my_abs(val):  
    if val < 0:  
        return 0 - val  
    return val  
  
print(val)
```

Explain the error in your own words.

Q8.2. Create a global variable `counter = 0`. Write a function `show_counter()` that prints `counter`. Call the function and confirm it can read the global variable.

Q8.3. Now write a function `increment_counter()` that tries to do `counter = counter + 1` without using the `global` keyword. Run it and explain the error you get.

Q8.4. Fix the function from the previous question using the `global` keyword so it correctly increments the global `counter` variable.

Scenario Based Questions

Q1. Shopping Cart Item Imagine you're helping build a shopping app. You need to store the name of one item.

- Step 1: Create a variable `item_name` and store the text "Notebook" in it.
- Step 2: Print the variable.
- Step 3: Now change `item_name` to store a number instead, like 101 (maybe this is now an item code).
- Step 4: Print it again and also print `type(item_name)` both times.

Expected output:

Notebook

```
<class 'str'>
```

101

```
<class 'int'>
```

What this teaches: the same variable can hold different types at different times — no errors, unlike C/Java.

Q2. Student ID Card You're creating one student's ID card details.

- Create `name` (text) = "Ananya"
- Create `roll_number` (whole number) = 21
- Create `height_in_cm` (decimal number) = 152.5
- Create `is_hostel_student` (True/False) = True
- Print all four values, each on its own line, with a label like **Name: Ananya**.

Expected output:

Name: Ananya

Roll Number: 21

Height: 152.5

Hostel Student: True

Q3. Comparing Two Friends' Ages Two friends, Sam and Priya, are both 20 years old.

- Create `sam_age = 20` and `priya_age = 20`.
- Print `sam_age == priya_age` — this checks if the *values* are equal.
- Print `sam_age is priya_age` — this checks if they are the *exact same object* in memory.
- Write one sentence explaining why `==` gives `True` even though they are two separate variables.

Expected output:

True

True

Note: for small numbers Python often reuses the same object, so both may show `True` — that's okay, the important part is understanding the difference in what each is checking.

Q4. Fixing a Messy Name A user types their name into a form like this: " rohan MEHTA " (extra spaces, wrong capitalization).

- Store this in a variable called `raw_name`.
- Use `.strip()` to remove the extra spaces.
- Use `.title()` to capitalize it properly (first letter of each word capital).
- Print the final cleaned-up name.

Expected output:

Rohan Mehta

Q5. Watering Plants Reminder You water your plants every day for 7 days. Write a `for` loop using `range()` that prints:

```
Day 1: Watered the plant
Day 2: Watered the plant
Day 3: Watered the plant
Day 4: Watered the plant
Day 5: Watered the plant
Day 6: Watered the plant
Day 7: Watered the plant
```

Hint: use `range(1, 8)` and an f-string like `print(f"Day {i}: Watered the plant")`.

Q6. Counting Down for a Rocket Launch Write a `while` loop that starts at 5 and counts down to 1, printing each number, and finally prints "Blast off!" after the loop ends.

Expected output:

```
5
4
3
2
1
Blast off!
```

Q7. Movie Ticket Price A cinema charges ₹100 for children (age below 12) and ₹250 for everyone.

- Create a variable `age = 10`.
- Write an `if/else` statement that prints the correct ticket price based on age.
- Test it by changing `age` to 15 and running again.

Expected output (for age = 10):

Ticket price: ₹100

Expected output (for age = 15):

Ticket price: ₹250

Q8. Even or Odd Roll Number Write a program that checks if a student's roll number is even or odd using %.

- Create `roll_number = 23`.
- If `roll_number % 2 == 0`, print "Even roll number", otherwise print "Odd roll number".

Expected output:

Odd roll number

Q9. Simple Greeting Function Write a function called `greet_student(name)` that takes a name and prints "Hello, <name>! Welcome to Python class."

- Call the function with "Meera" and then with "Karan".

Expected output:

Hello, Meera! Welcome to Python class.

Hello, Karan! Welcome to Python class.

Q10. Add Two Numbers Write a function `add_numbers(a, b)` that **returns** the sum of `a` and `b` (don't just print it inside the function — return it, then print it outside).

- Call `add_numbers(4, 7)` and print the result.

Expected output:

11

Q11. Class Attendance Counter

- Create a global variable `total_students = 0`.
- Write a function `mark_present()` that increases `total_students` by 1 — remember to use the `global` keyword inside the function so it can change the global variable.
- Call `mark_present()` three times (once for each student who walks in), then print `total_students`.

Expected output:

3